

# Behavioral Prompt Compression: Objective and Architecture

## Prompt Compression Layer

### 1 Problem

Let  $M$  be a frozen target model configuration,  $P$  an original prompt template, and  $X = \{x_i\}_{i=1}^n$  an input set. The compiler searches for a compressed prompt template  $P'$  that reduces instruction tokens while preserving target-model behavior:

$$M(P', x_i) \approx M(P, x_i) \quad \text{for many } x_i \in X.$$

Reference outputs are behavioral references:

$$y_i = M(P, x_i).$$

Candidate outputs are:

$$\hat{y}_{i,c} = M(P'_c, x_i).$$

### 2 Token Objective

Instruction-token reduction is measured on the invariant prompt layer:

$$r_c = 1 - \frac{T_{\text{instr}}(P'_c)}{T_{\text{instr}}(P)}.$$

Full rendered prompt tokens can also be tracked:

$$\bar{T}_{\text{full}}(P'_c) = \frac{1}{n} \sum_{i=1}^n T(P'_c(x_i)).$$

### 3 Embedding Drift

For an embedding model  $E$ , embedding drift uses Euclidean distance:

$$d_{\text{emb}}(c, i) = \|E(\hat{y}_{i,c}) - E(y_i)\|_2.$$

The average embedding drift is:

$$\bar{d}_{\text{emb}}(c) = \frac{1}{n} \sum_{i=1}^n d_{\text{emb}}(c, i).$$

## 4 Hybrid Loss

The candidate loss is a weighted behavioral objective:

$$\mathcal{L}(c) = \lambda_t \rho_c + \lambda_e \bar{d}_{\text{emb}}(c) + \lambda_j \bar{d}_{\text{judge}}(c) + \lambda_f \phi_c + \lambda_q \tau_c + \lambda_v \nu_c,$$

where:

$$\rho_c = \frac{T_{\text{instr}}(P'_c)}{T_{\text{instr}}(P)}$$

is the token ratio,  $\phi_c$  is the format or schema failure rate,  $\tau_c$  is the task-metric failure rate, and  $\nu_c$  is output variance across repeated runs.

Contract failures can be hard penalties:

$$\mathcal{L}(c) \leftarrow \mathcal{L}(c) + 10 \quad \text{if a required output contract fails.}$$

## 5 Pareto Frontier

Candidate  $a$  dominates candidate  $b$  when:

$$r_a \geq r_b, \quad \bar{d}_a \leq \bar{d}_b, \quad \phi_a \leq \phi_b, \quad \tau_a \leq \tau_b$$

and at least one inequality is strict.

The compiler returns the non-dominated set:

$$\mathcal{F} = \{c \in C : \nexists c' \in C \text{ such that } c' \succ c\}.$$

## 6 System Layers

| Layer        | Purpose                             | Examples                          |
|--------------|-------------------------------------|-----------------------------------|
| Chunking     | Split invariant prompt instructions | schema, role, task, negation      |
| Exploration  | Rewrite and assemble candidates     | DSL, Mandarin, compact English    |
| Evaluation   | Compare behavior across inputs      | Euclidean drift, contracts, judge |
| Optimization | Improve candidates over epochs      | Pareto, credit, mutation          |